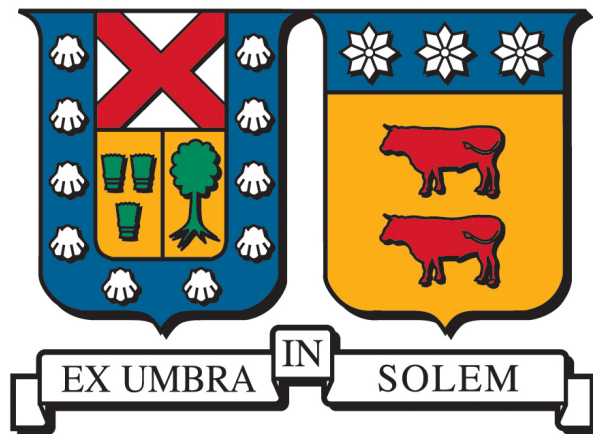




Entrega 1 - Directorio Médico

Debora Alayo Azabache
Ernesto Barría Andrade
Joaquín Gatica Hernández
Sebastián Gutiérrez Milla

Repositorio: <https://github.com/Equipo2PDW/ProyectoDirectorioMedico>

Video: <https://youtu.be/xwm8K5Wm9MQ>



Índice

1. Metodología de trabajo	2
2. Alcance de la herramienta	3
3. Descripción del trabajo realizado	3
3.1. Proyecto	3
3.2. Especificar dependencias entre la herramienta y la aplicación	3
4. Pruebas	3
4.1. Estrategias de Pruebas utilizadas	4
4.1.1. Objetivo de las pruebas	4
4.1.2. Alcance de las pruebas	4
4.1.3. Tipos de pruebas	4
4.1.4. Ambiente de pruebas	4
4.2. Procedimiento de ejecución de pruebas	4
4.3. Resultados de las pruebas	5
5. Problemas encontrados y soluciones	5

1. Metodología de trabajo

En este trabajo utilizamos la metodología Kanban mediante la aplicación Jira, la cual tiene integraciones con GitHub. Para manejar el repositorio utilizamos Gitflow.

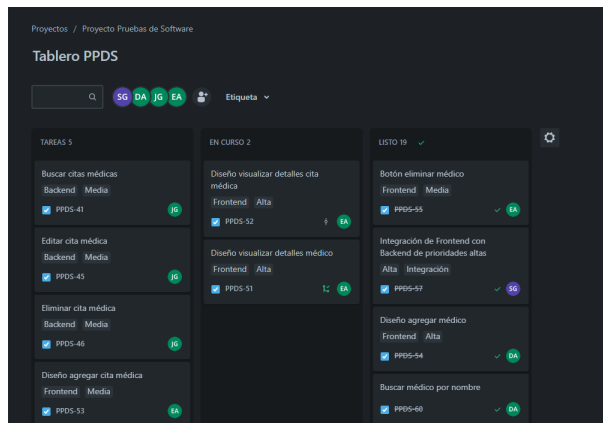


Gráfico 1: Tablero de Jira con nuestros requisitos.

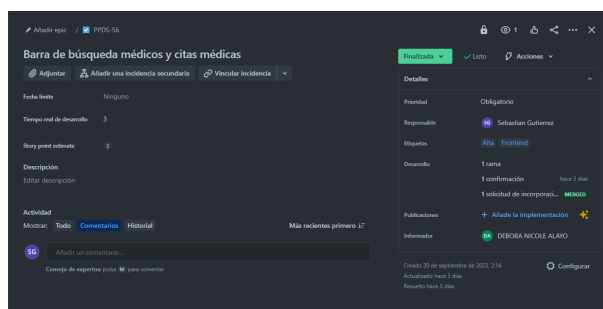


Gráfico 2: Incidencia de Jira con story points e integración de GitHub.

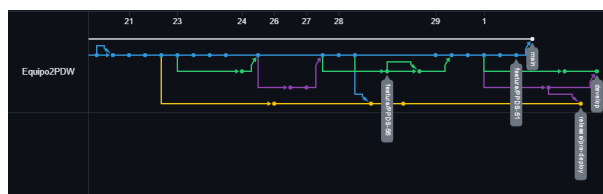


Gráfico 3: Parte del network graph del repositorio con las branch definidas para usar Gitflow.



2. Alcance de la herramienta

Robot es un framework de Python diseñado específicamente para la automatización de pruebas de aceptación. Durante el proceso de desarrollo de pruebas, se hizo evidente este enfoque, ya que se volvía significativamente más complicado crear pruebas unitarias en comparación con la prueba conjunta del backend y el frontend. Por esta razón, a pesar de encontrarnos en las primeras etapas del proyecto, aún no hemos implementado pruebas unitarias ni pruebas de integración. En su lugar, hemos optado por centrarnos en el tipo de pruebas para el cual el framework está orientado, esto nos permite validar la funcionalidad completa de la aplicación desde la perspectiva del usuario final, lo cual no es tan fundamental en este punto del desarrollo.

3. Descripción del trabajo realizado

3.1. Proyecto

Para la creación de este proyecto, se eligió trabajar con Django para el backend y frontend, Jinja como lenguaje de templates, Figma para diseñar la página y Robot para hacer las pruebas.

3.2. Especificar dependencias entre la herramienta y la aplicación

El framework robot es el encargado de interactuar con la UI mediante código de Python y Django es el encargado de crear frontend y backend y hacerlos interactuar entre sí.

4. Pruebas

La versión temprana de la test suite se compone de 6 pruebas de aceptación:

- **[TA-01] Crear médico:** Esta prueba consiste en ingresar a la vista, la cual contiene el formulario con todos los datos que un médico debe tener, lo rellenamos y presionamos el botón para crear el médico. Para saber si la prueba fue satisfactoria nos guiamos por el mensaje que manda la página al crear un médico correctamente.
- **[TA-02] Actualizar información médico:** Esta prueba consiste en ingresar a la vista mediante el URL con la id del médico a editar, la cual contiene un formulario con la información actual de este. Editamos y presionamos el botón para actualizar. Para saber si la prueba fue satisfactoria nos guiamos por la vista actual, muestra al médico con la información que le proporcionamos.
- **[TA-03] Buscar por Comuna:** Esta prueba consiste en ingresar a la vista, la cual contiene un dropdown con las comunas únicas de todos los médicos inscritos en nuestra base de datos. Elegimos una comuna y presionamos el botón para buscar. Para saber si la prueba fue satisfactoria nos guiamos por todos los doctores que aparecen en la búsqueda tienen la comuna elegida asociada.
- **[TA-04] Buscar por Especialidad:** Esta prueba consiste en ingresar a la vista, la cual contiene un dropdown con las especialidades únicas de todos los médicos inscritos en nuestra base de datos. Elegimos una especialidad y presionamos el botón para buscar. Para saber si la prueba fue satisfactoria nos guiamos por todos los doctores que aparecen en la búsqueda tienen la especialidad elegida asociada.



- [TA-05] **Buscar por Nombre:** Esta prueba consiste en ingresar a la vista, la cual contiene un recuadro para ingresar el nombre del médico, lo ingresamos y presionamos el botón para buscar. Para saber si la prueba fue satisfactoria nos guiamos por todos los doctores que aparecen en la búsqueda tienen el nombre que elegimos.
- [TA-06] **Eliminar médico:** Esta prueba consiste en ingresar a la vista mediante el URL con la id del médico para ver sus detalles. Presionamos el botón para eliminar, y confirmamos. Para saber si la prueba fue satisfactoria nos guiamos porque al eliminar el médico, nos redirige a la página para buscar un especialista.

4.1. Estrategias de Pruebas utilizadas

4.1.1. Objetivo de las pruebas

En este caso, por el enfoque de la herramienta que utilizamos, nos queríamos enfocar en probar el funcionamiento de la aplicación con un "flujo correcto". Por lo que solamente hicimos pruebas teniendo en cuenta de que el usuario realiza las acciones correctamente y no se hicieron pruebas probando los límites de cada componente.

Para futuras entregas, sumado a una mayor experiencia con el framework se espera poder hacer la suitcase más robusta comparada a esta primera entrega.

4.1.2. Alcance de las pruebas

Dicho esto, las partes que están siendo sometidas a prueba serían todas las funciones encargadas del CRUD para los especialistas y las vistas que estén relacionadas.

4.1.3. Tipos de pruebas

Hasta el momento, hemos optado por llevar a cabo pruebas de aceptación debido a que el framework que estamos utilizando está altamente orientado a este tipo de pruebas.

4.1.4. Ambiente de pruebas

El repositorio contiene un archivo "db.sqlite3".^{el} cual ya contiene médicos agregados en la base de datos, los cuales serán ocupados para realizar las pruebas.

4.2. Procedimiento de ejecución de pruebas

La test suitcase está programada en un solo archivo, por lo que, solo con la ejecución de este, se realizaran todas las pruebas pertinentes.



4.3. Resultados de las pruebas

Las 6 pruebas tuvieron resultados satisfactorios.

Scenario: Crear medico	PASS
Scenario: Actualizar informacion medico	PASS
Scenario: Buscar por Comuna	PASS
Scenario: Buscar por Especialidad	PASS
Scenario: Buscar por Nombre	PASS
Scenario: Eliminar medico	PASS
Test :: Django Robot Tests	PASS
6 tests, 6 passed, 0 failed	

Gráfico 4: La terminal mostrando los resultados de las pruebas.

5. Problemas encontrados y soluciones

A pesar de tener dificultades para usar un nuevo framework, al momento de programar el frontend, las etiquetas de HTML no tenían la propiedad de id, pero el framework robot los necesitaba, por lo que tuvimos que revisar todos los templates y agregar la propiedad que faltaba.